MP-1 CE-MATH

Group 7: Bilanes, Catapang, Obar

The project revolves around creating a program that would compute the area of the function y = e^(dt) * sin(ct) using numerical integration (i.e, Trapezoidal Rule and Simpson's Rule) as well as finding the solution of the function using the analytic method to compare the answer with the result of the program

Solution of y = e^(dt) * sin(ct) using analytic method

To get the integral of the function, the group used integration by parts twice. When performing integration by parts the first time, the result would be:

I = - (cos(ct) *e^(dt))/c + (d/c) * ∫e^(dt) * cos(ct)dt

Getting the integral of our resulting integration will give us the following:

J = (sin(ct) * e^(dt))/c - (d/c) * ∫e^(dt) * sin(ct)dt

Since (∫e^(dt) * cos(ct)dt) in I is equal to J and (∫e^(dt) * sin(ct)dt) in J is equal to I, we can perform substitution so I will become:

I = (sin(ct) *e^(dt))/c - (d/c) * ((cos(ct) * e^(dt)) - (d/c) * I),

Simplify:

I = (cos(ct) *e^(dt))/c + (de^(dt) * sin(ct))/c^2 - (d^2/c^2) * I

Combining like terms we get:

I = (e^(dt) * dsin(ct) - ccos(ct)) / (c^2 + d^2)

Full solution below:

$$\int_a^b e^{dt} \sin(ct)\, dt$$

$$u = e^{dt} \qquad du = d\, e^{dt}\, dt$$

$$dv = \sin(ct)\, dt \qquad v = -\frac{\cos(ct)}{c}$$

$$= uv - \int v\, du = e^{dt}\left(-\frac{\cos(ct)}{c}\right) - \int\left(-\frac{\cos(ct)}{c}\right) d e^{dt}\, dt$$

$$= -\frac{e^{dt}\cos(ct)}{c} + \frac{d}{c}\int e^{dt}\cos(ct)\, dt$$

$$\int e^{dt}\cos(ct)\, dt = J = -\frac{e^{dt}\cos(ct)}{c} + \frac{d}{c} J$$

$$(uv - \int u\, dv)$$

$$= J = \int e^{dt}\cos(ct)\, dt = \frac{e^{dt}\sin(ct)}{c} - \frac{d}{c}\int e^{dt}\sin(ct)\, dt$$

$$u = e^{dt} \qquad du = d\, e^{dt}\, dt$$

$$dv = \cos(ct)\, dt \qquad v = \frac{\sin(ct)}{c} = \frac{e^{dt}\sin(ct)}{c} - \frac{d}{c} I$$

$$I = -\frac{e^{dt}\cos(ct)}{c} + \frac{d}{c}\left(\frac{e^{dt}\sin(ct)}{c} - \frac{d}{c} I\right)$$

$$= -\frac{e^{dt}\cos(ct)}{c} + \frac{d\, e^{dt}\sin(ct)}{c^2} - \frac{d^2}{c^2} I$$

$$I\left(1 + \frac{d^2}{c^2}\right) = e^{dt}\left(-\frac{\cos(ct)}{c} + \frac{d\sin(ct)}{c^2}\right)$$

$$\frac{c^2 + d^2}{c^2} \rightarrow I = \frac{c^2}{c^2 + d^2}\, e^{dt}\left(-\frac{\cos(ct)}{c} + \frac{d\sin(ct)}{c^2}\right)$$

$$\int_a^b e^{dt}\sin(ct)\, dt = \boxed{\left[\frac{e^{dt}(d\sin(ct) - c\cos(ct))}{c^2 + d^2}\right]_a^b}$$

Writing the program

For the following user input statements, we've used while loops, try and catch, and if-else statements to keep asking the user to encode 6 values. M for the of method (Trapezoidal or Simpson's rule), c & d as constants, a & b as lower and upper limits of integration and n for number of intervals. We've also made some additional variables such as v which is used to determine if we can end each while loop, the variable areas to store the segment areas and reserved which contains the reserved variable names to prevent the user from inputting them. We've also used some built in functions such as str2double(), eva()l, isnan(), isempty() and mode to further validate the user inputs.

Validate User Input for Method Selection

```C/C++
reserved = {'a','b','c','d','n','M','v','t','y','dt','reserved',
'areas'};
v = 0
areas = [];
while v == 0
    M = input("Choose an option: \n'1' - for Trapezoidal Rule\n'2' - for
Simpson's Rule\n\nWhat is your choice?: ", 's');
    M = str2double(M); % converts '1' into 1 and 'a' into nan
    if isnan(M) || isempty(M) % checks if value is NaN or ''
        disp('Invalid Input!');
    elseif M == 1
       disp('Trapezoidal Rule chosen');
       v = 1;
    elseif M == 2
       disp("Simpson's Rule chosen");
       v = 1;
    else
       disp('Invalid Input!');
    end
end
```

Validate User Input for Values of c & d

```C/C++
v = 0;
% user value inputs
while v == 0
```

```
    c = input('Enter value of c for y = e^(dt)*sin(ct) function: ',
's');
    d = input('Enter value of d for y = e^(dt)*sin(ct) function: ',
's');
    if any(strcmp(c, reserved)) || any(strcmp(d, reserved))
        disp('Invalid Input!');
    else
        try % converts numerical strings such as '1', 'pi' into
numerical values
            c = eval(c);
            d = eval(d);
        catch % if non-numerical string, change the value to NaN
            c = NaN;
            d = NaN;
        end
        if (isnan(c) || isempty(c)) || (isnan(d) || isempty(d)) % checks
if value is NaN or ''
            disp('Invalid Input!');
        else
            v = 1;
        end
    end
end
```

Validate User Input for Lower and Upper Limits

```C/C++
v = 0;
while v == 0
    a = input('Enter lower limit for the integration: ', 's');
    b = input('Enter upper limit for the integration: ', 's');
    if any(strcmp(a, reserved)) || any(strcmp(b, reserved))
        disp('Invalid Input!');
    else
        try % converts numerical strings such as '1', 'pi' into
numerical values
            a = eval(a);
            b = eval(b);
        catch % if non-numerical string, change the value to NaN
```

```
            a = NaN;
            b = NaN;
        end
        if (isnan(a) || isempty(a)) || (isnan(b) || isempty(b)) % checks
if value is NaN or ''
           disp('Invalid Input!');
        elseif(a > b)
           disp('Lower limit should be less than upper limit.')
        else
           v = 1;
        end
    end
end
```

Validate User Input for Number of Intervals

```
C/C++
v = 0;
while v == 0
    n = input('Enter number of intervals: ', 's');
    if any(strcmp(n, reserved))
        disp('Invalid Input!');
    else
        try % converts numerical strings such as '1', 'pi' into numerical
values
            n = eval(n);
        catch % if non-numerical string, changes the value to NaN
            n = NaN;
        end
         if isnan(n) || isempty(n) || n <= 0 || mod(n,1) ~= 0 % checks if value
is NaN, '',is not positive or is a float
                disp('Invalid Input!')
         elseif M == 2 && mod(n,2) ~= 0 % in simpson's rule, interval must be
even
                disp('Interval must be even.');
         else
                v = 1;
         end
    end
end
```

Over here is where we compute for delta t or more commonly known as Δx, we also compute for t which are the values from a to b and y stores the values of f(a) to f(b). The variable zc on the other hand stores the indices of y where the function crosses zero while t(zc) are the x values of the zero crossings. After this, we can finally compute for the segment areas and the total area using trapezoidal or simpson's rule.

Points of Zero Crossings

```
C/C++
dt = (b - a) / n; % compute delta t
t = a:dt:b;               % t-values
y = exp(d*t) .* sin(c*t); % function values with t-values

% zero crossing points
zc = find(diff(sign(y)));  % finds indices where y changes sign
t(zc); %t-values of the zero crossing points of y

% displays the zero crossing points
fprintf('Zero crossings at t = ');
fprintf('%.4f ', t(zc));
fprintf('\n');


% areas under the curve
% start and end points of zero crossings
zc_full = [1, zc, length(t)];
total_area = 0;
part = 1;
```

Trapezoidal Rule

```
C/C++
for k = 1 : length(zc_full)-1
    t_part = t( zc_full(k) : zc_full (k+1) );
    y_part = y( zc_full(k) : zc_full (k+1) );

    if numel(t_part) < 2 % ensures so that program continues if t_part <
2
        continue;
    end
```

```
    dt_part = t_part(2) - t_part(1);

    n_part = length(t_part) - 1; % number of subintervals in this
segment

    if M == 1 % if trapezoidal rule

        % first and last ordinates
        first = y_part(1);
        last = y_part(end);

        % sum of remaining ordinates
        remaining_sum = sum(y_part(2:end-1));

        % Trapezoidal Rule formula
        area = dt_part * ((0.5 * (first + last)) + remaining_sum);
```

Simpsons Rule & Total Area

```C/C++
elseif M == 2 % if simpsons rule
            % first and last ordinates
            first = y_part(1);
            last = y_part(end);

            % odd and even sum of ordinates
            even_sum = sum(y_part(3:2:end-2));
            odd_sum  = sum(y_part(2:2:end-1));

            % Simpsons rule formula
            area = (dt_part / 3) * (first + last + 4*odd_sum +
2*even_sum);

    end
    % get absolute area
    area = abs(area);

    fprintf('Area for part %d: %.5f\n', part, area);
```

```
    areas(end+1) = area;
    total_area = total_area + area;
    part = part + 1;
end

fprintf('Total area under the Curve: %.4f\n', total_area)
```

Plotting and Labeling of x and y axis

```C/C++
figure;
% t and y are the x and y axis
% b for blue and - solid line and 2 for thickness
plot(t, y, 'b-', 'LineWidth', 2); hold on; grid on;

% mark zero crossing
plot(t(zc), y(zc), 'ro', 'MarkerFaceColor','r');
xlabel('x');
ylabel('e^(dt)*sin(ct)');
title('Area Under the Curve of y = e^{dt} * sin(ct)');
```

Graphing and Shading (Full Graph)

```C/C++
%shaded areas

zc = zc(zc > 1 & zc < length(t));
shade = [1,zc,length(t)];

for k = 1:length(shade)-1
    if shade(k+1) <= shade(k)
        continue;
    end

    idx = shade(k):shade(k+1);
    if idx(1) < 1 || idx(end) > length(t)
        continue;
    end
```

```
    % Create shaded area
    fill([t(idx), fliplr(t(idx))], [y(idx), zeros(size(y(idx)))], 'b',
'FaceAlpha', 0.3, 'EdgeColor', 'none');

end

for k = 1:length(shade)-1
    if shade(k+1) <= shade(k)
        continue;
    end
    idx = shade(k):shade(k+1);
    if isempty(idx)
        continue;
    end
    % plot each area separately
    figure;
    plot(t(idx), y(idx), 'b-', 'LineWidth', 2); hold on; grid on;
    plot(t(idx([1 end])), y(idx([1 end])), 'ro', 'MarkerFaceColor','r');
    %shade each area
    fill([t(idx), fliplr(t(idx))], [y(idx), zeros(size(y(idx)))], 'b',
'FaceAlpha', 0.3, 'EdgeColor', 'none');
    xlabel('x');
    ylabel('e^{dt} * sin(ct)');
    title(sprintf('Area = %.5f', areas(k)));
end
```

Graphing and Shading (Individual Areas of the Graph)

C/C++

```
for k = 1:length(shade)-1
    if shade(k+1) <= shade(k)
        continue;
    end
    idx = shade(k):shade(k+1);
    if isempty(idx)
        continue;
    end
    % plot each area separately
```

```
    figure;
    plot(t(idx), y(idx), 'b-', 'LineWidth', 2); hold on; grid on;
    plot(t(idx([1 end])), y(idx([1 end])), 'ro', 'MarkerFaceColor','r');
    %shade each area
    fill([t(idx), fliplr(t(idx))], [y(idx), zeros(size(y(idx)))], 'b',
'FaceAlpha', 0.3, 'EdgeColor', 'none');
    xlabel('x');
    ylabel('e^{dt} * sin(ct)');
    title(sprintf('Area = %.5f', areas(k)));
end
```

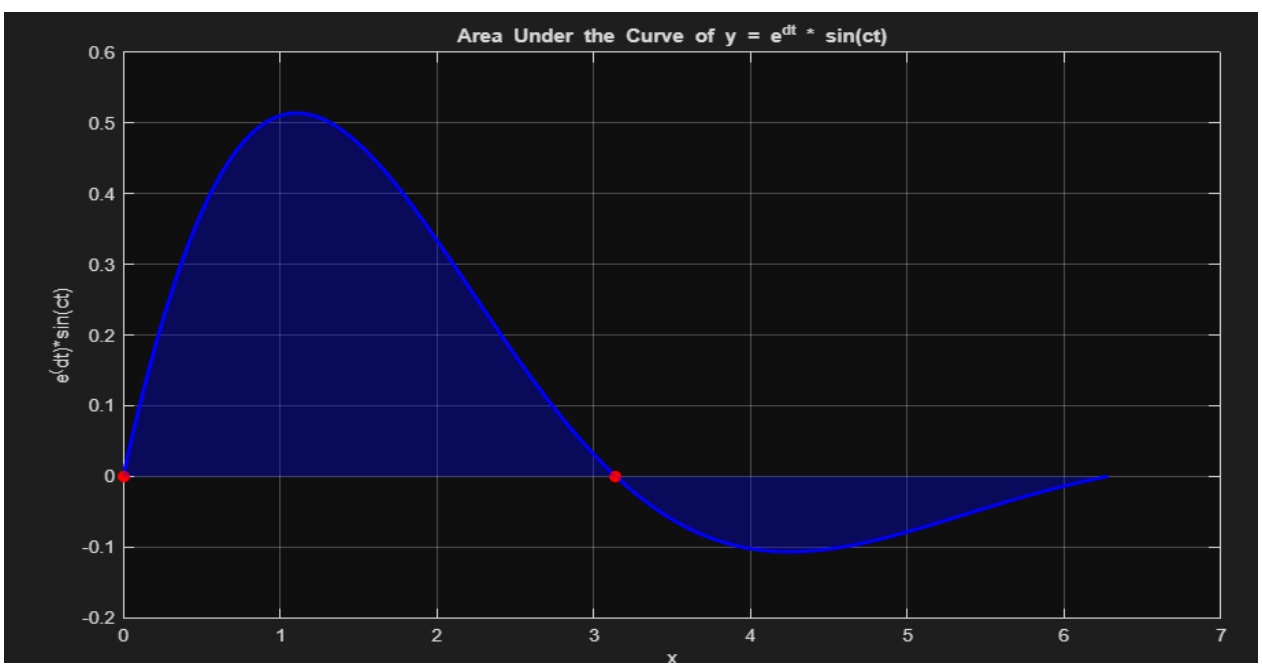
After evaluating the segment areas and the total area, we plot them into their own graphs. The first plot after figure draws the line for each segment area or for the total area, the second plot places a red dot at the endpoints of each segment and the fill function shades the area in the graph, this process is done with for loops to display and shade each segment and total area.

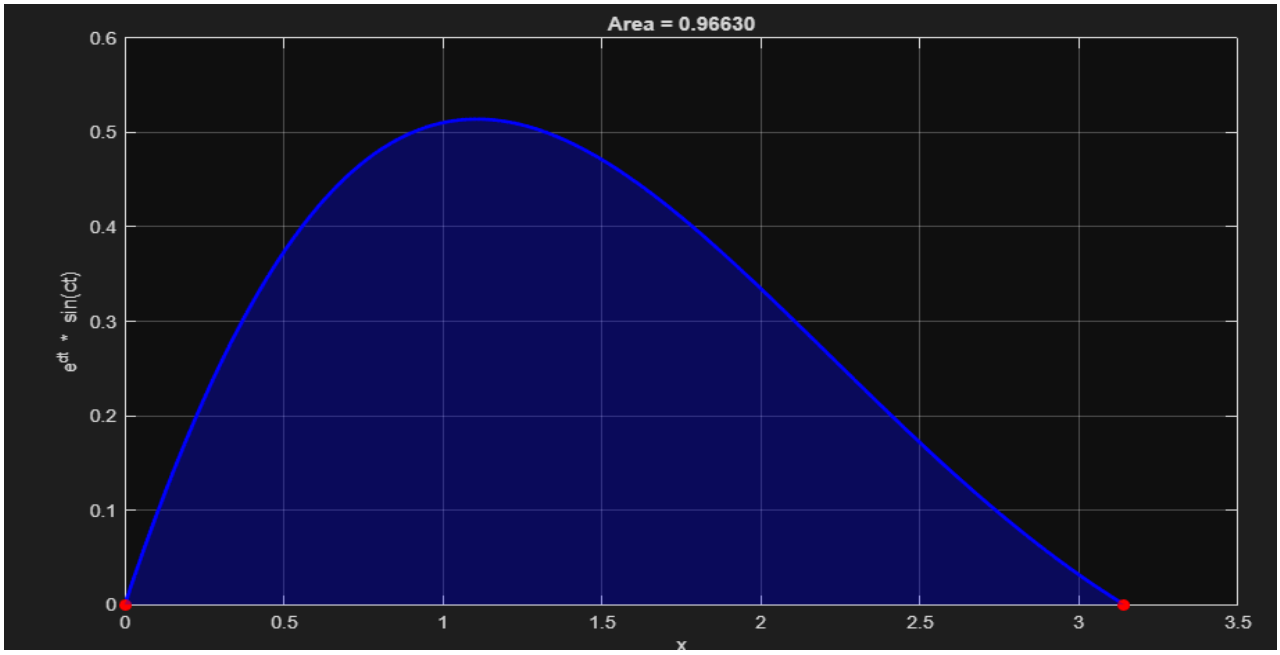
Output of the Program

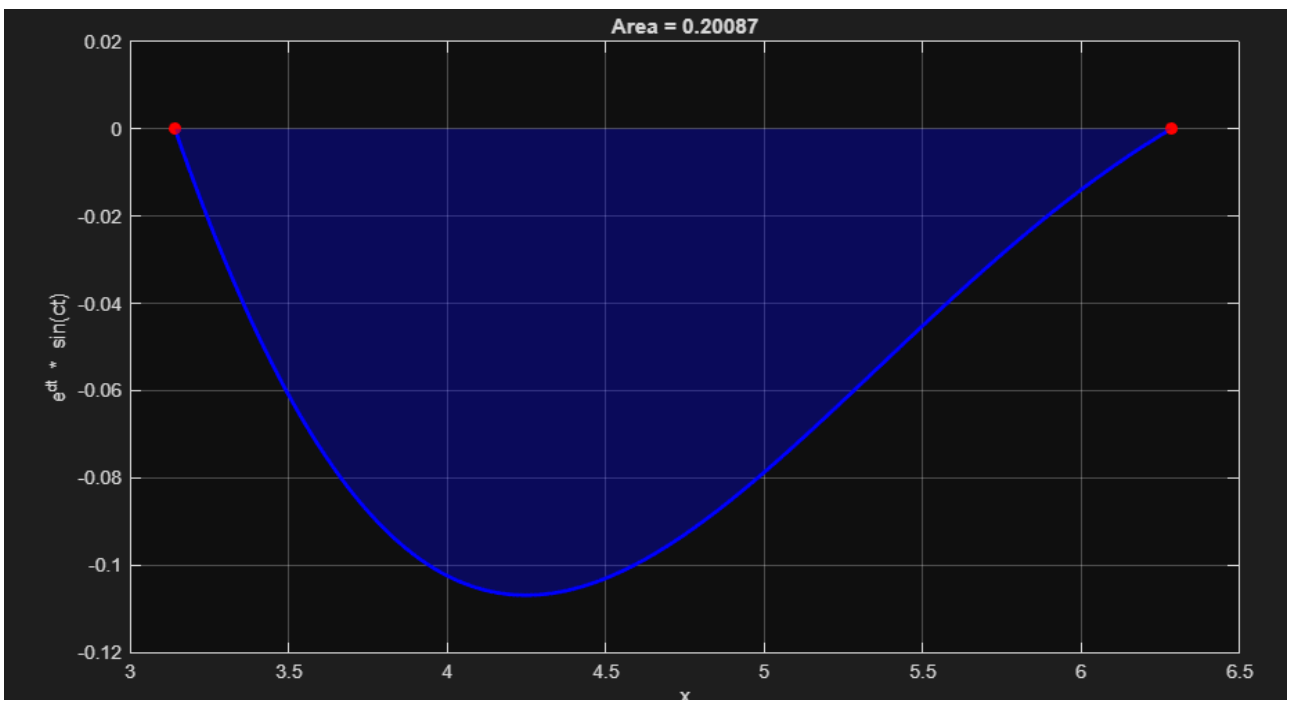
Using **Example 3** from the provided PDF:

Results:

```
This program computes the area under the curve of a function y =
e^(dt)*sin(ct) using a chosen numerical method.

Choose an option:
'1' - for Trapezoidal Rule
'2' - for Simpson's Rule

What is your choice?: 1
Trapezoidal Rule chosen
Enter value of c for y = e^(dt)*sin(ct) function: 1
Enter value of d for y = e^(dt)*sin(ct) function: -0.5
Enter lower limit for the integration: 0
Enter upper limit for the integration: 2*pi
Enter number of intervals: 1000
Zero crossings at t = 0.0000 3.1416
Area for part 1: 0.96630
Area for part 2: 0.20087
Total area under the Curve: 1.1672
```



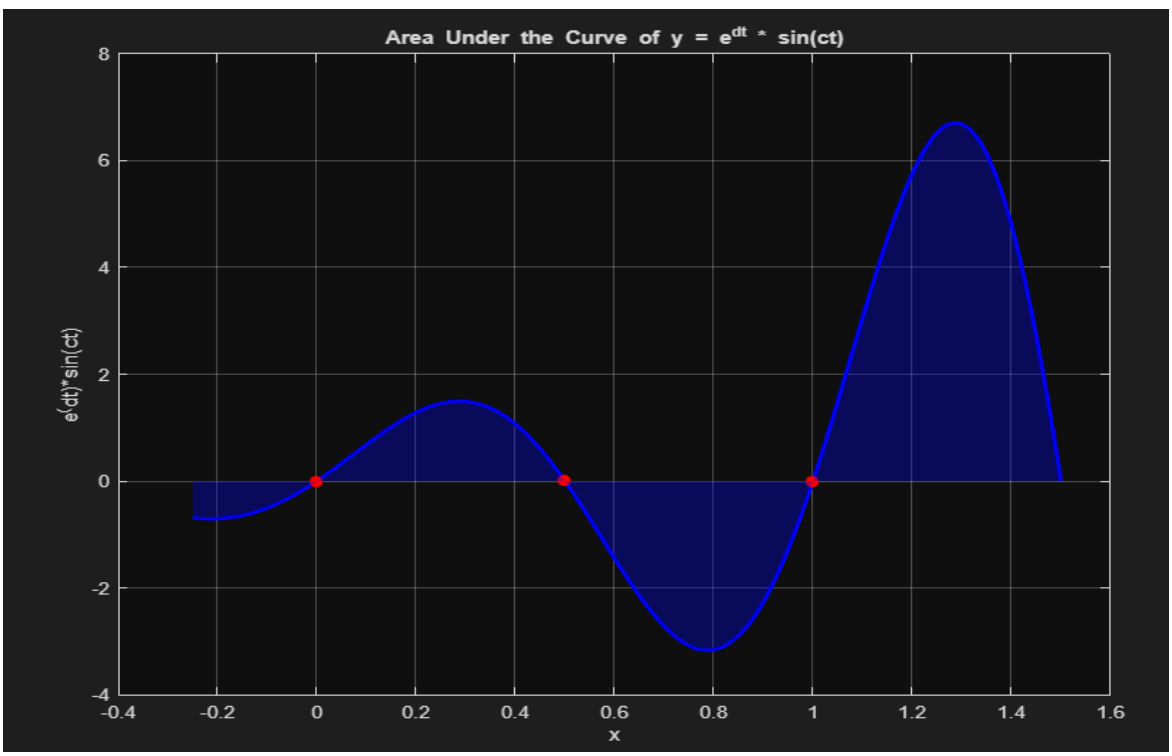

Area = 0.96630
0.6
0.5
0.4
0.3
0.2
0.1
0
0
0.5
1
1.5
2
2.5
3
3.5
x
e^dt * sin(ct)

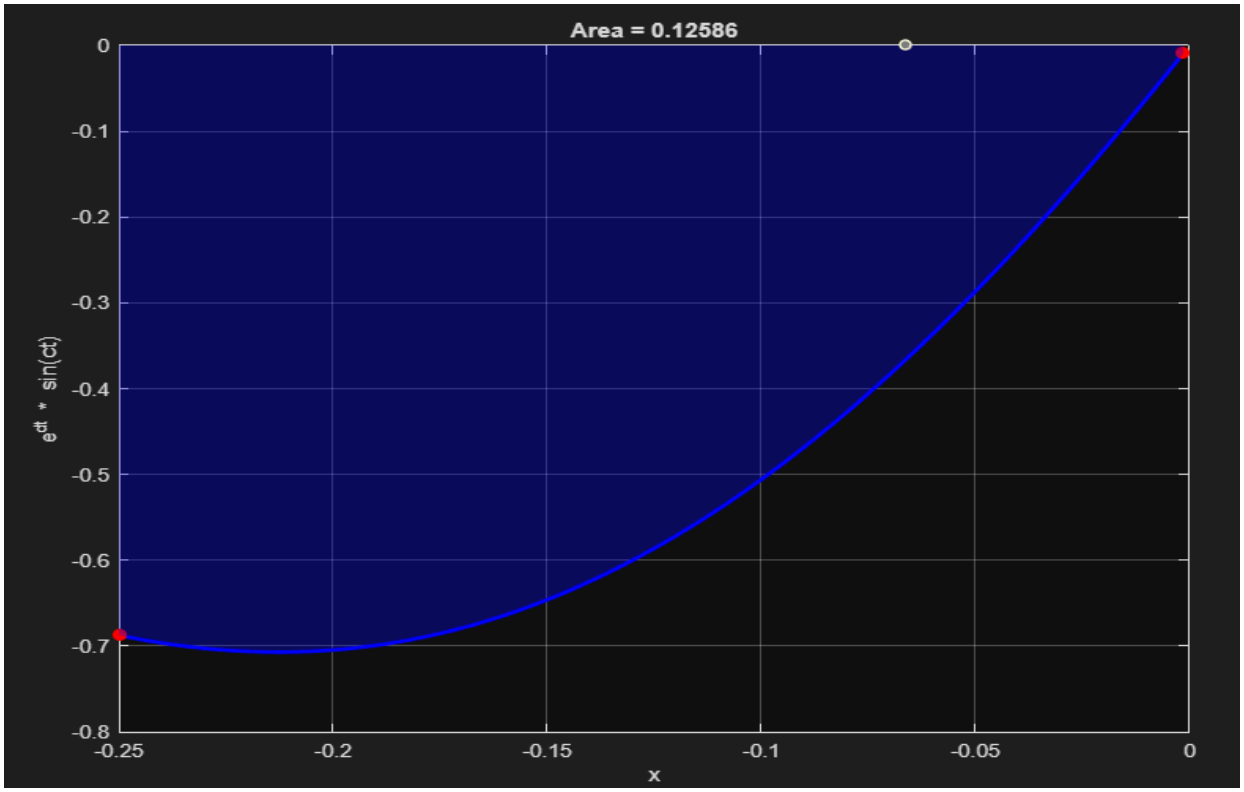

Area = 0.20087
0.02
0
-0.02
-0.04
-0.06
-0.08
-0.1
-0.12
3
3.5
4
4.5
5
5.5
6
6.5
x
e^dt * sin(ct)

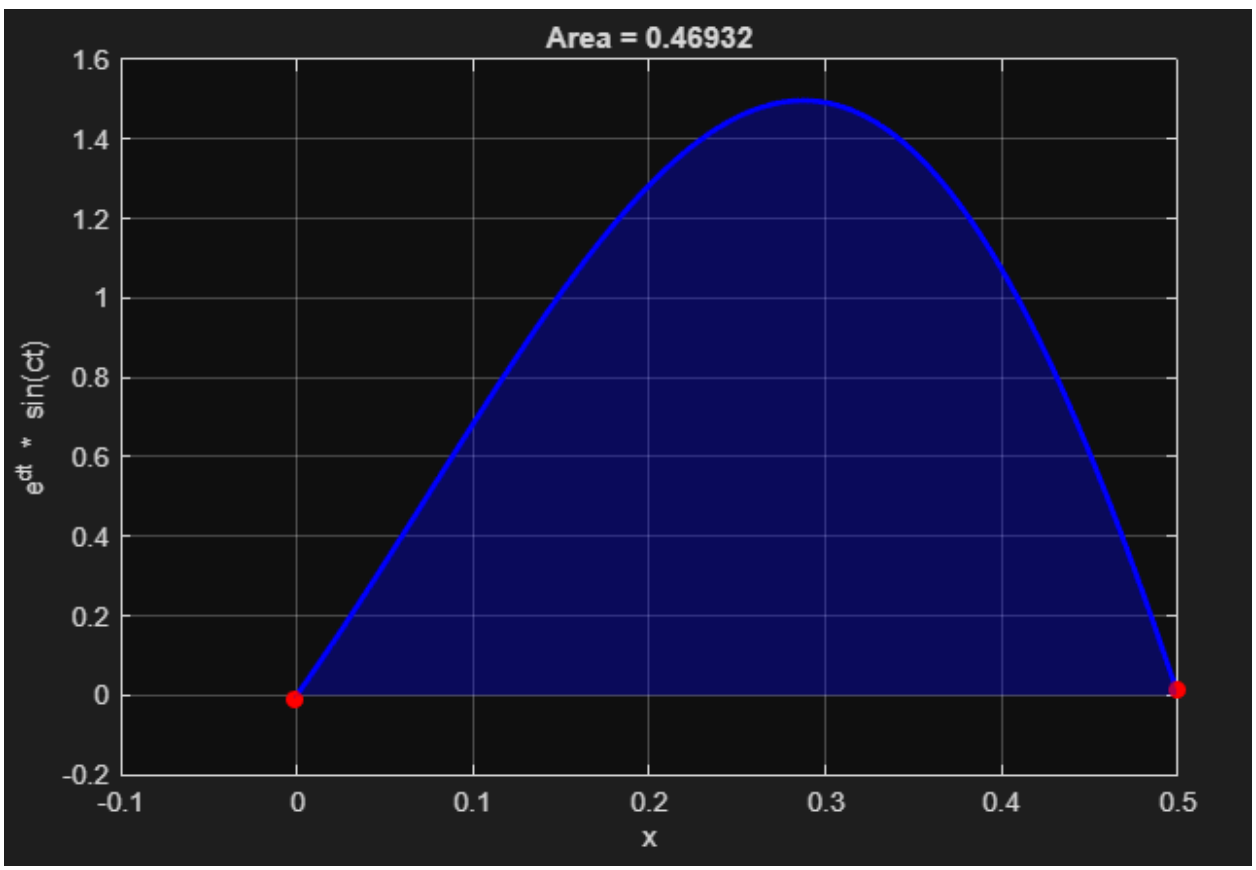
Using **Example 4** from the provided PDF:

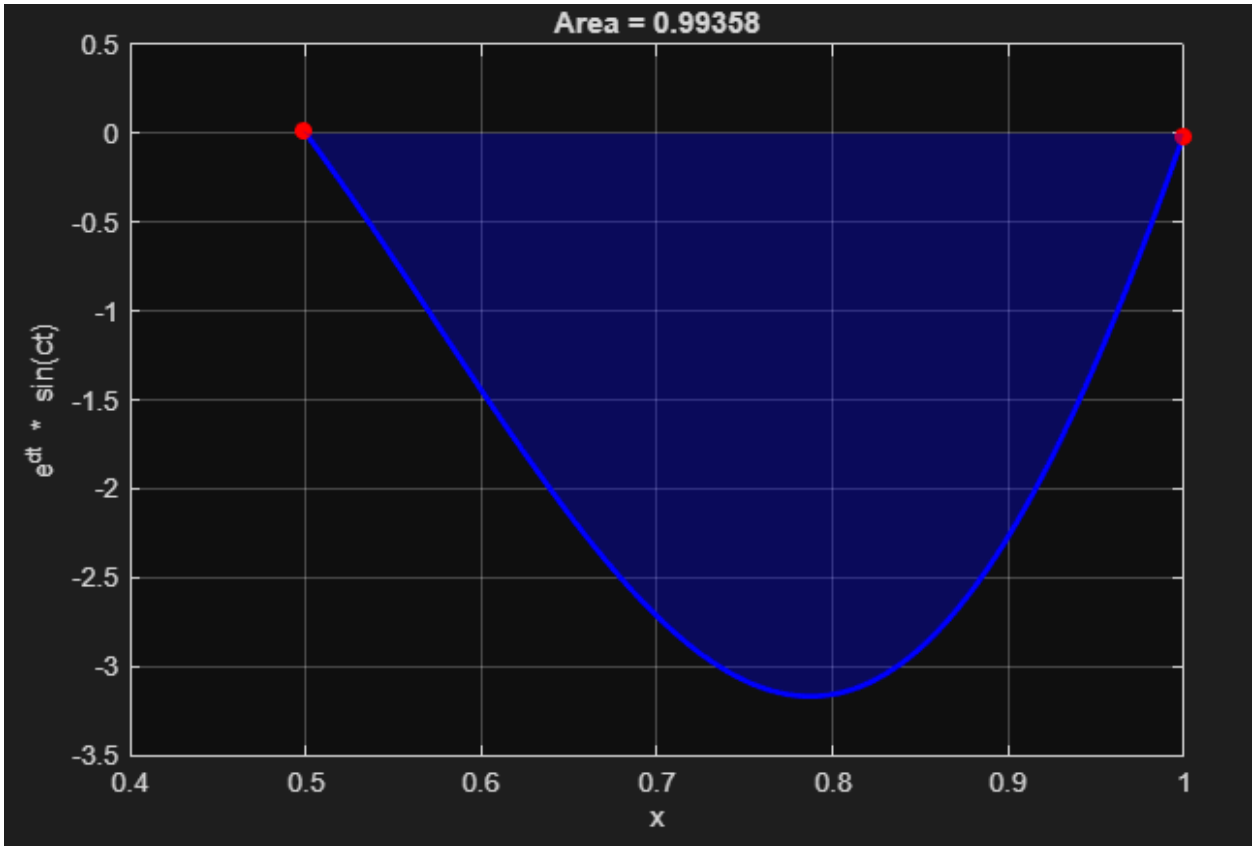
Results:

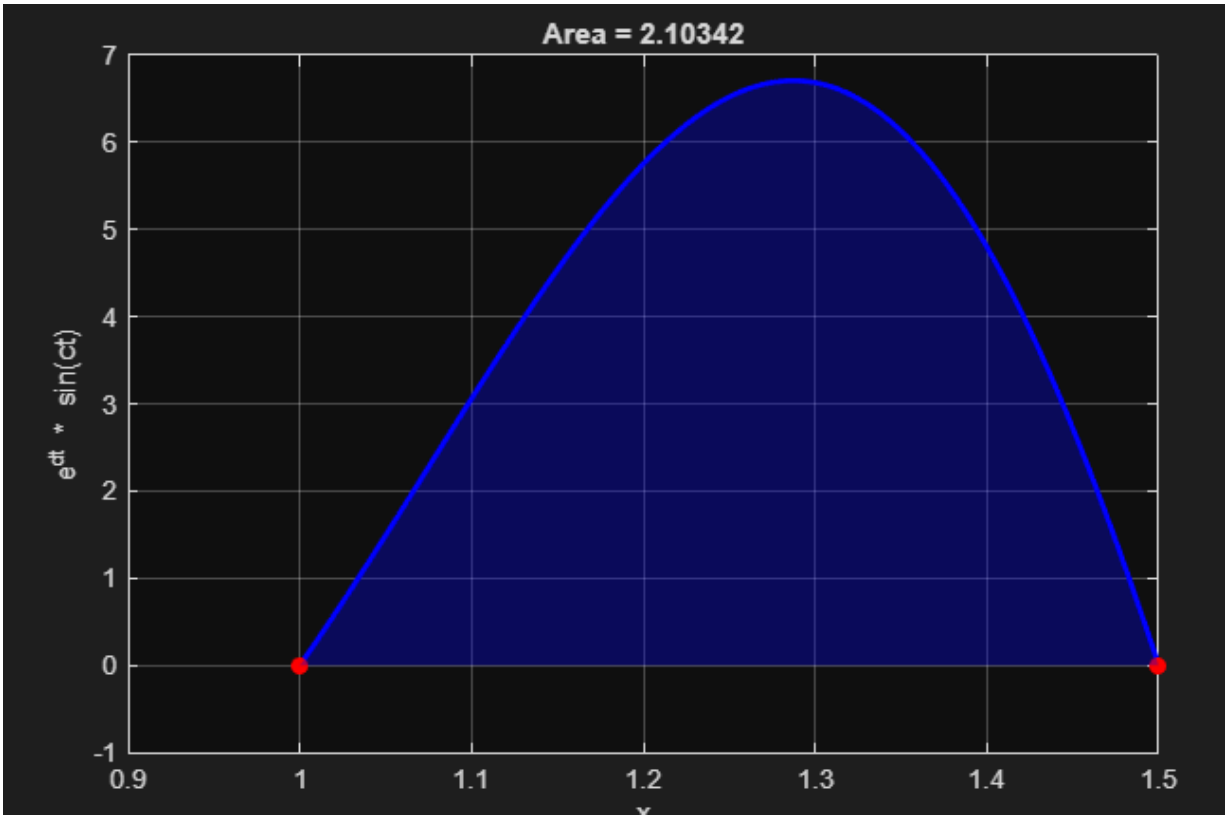
```
This program computes the area under the curve of a function y =
e^(dt)*sin(ct) using a chosen numerical method.

Choose an option:
'1' - for Trapezoidal Rule
'2' - for Simpson's Rule

What is your choice?: 2
Simpson's Rule chosen
Enter value of c for y = e^(dt)*sin(ct) function: 2*pi
Enter value of d for y = e^(dt)*sin(ct) function: 1.5
Enter lower limit for the integration: -0.25
Enter upper limit for the integration: 1.5
Enter number of intervals: 1000
Zero crossings at t = -0.0015 0.4990 0.9995
Area for part 1: 0.12586
Area for part 2: 0.46932
Area for part 3: 0.99358
Area for part 4: 2.10342
Total area under the Curve: 3.6922
```




Area = 0.12586
0
-0.1
-0.2
-0.3
-0.4
-0.5
-0.6
-0.7
-0.8
e^dt * sin(ct)
-0.25
-0.2
-0.15
-0.1
-0.05
0
x


Area = 0.46932
1.6
1.4
1.2
1
0.8
0.6
0.4
0.2
0
-0.2
e^dt * sin(ct)
-0.1
0
0.1
0.2
0.3
0.4
0.5
x


Area = 0.99358
0.5
0
-0.5
-1
-1.5
-2
-2.5
-3
-3.5
0.4
0.5
0.6
0.7
0.8
0.9
1
x
e^dt * sin(ct)


Area = 2.10342
7
6
5
4
3
2
1
0
-1
0.9
1
1.1
1.2
1.3
1.4
1.5
e^dt * sin(ct)

Error Handling:

- **Main Menu** : Values aside from 1 or 2
  Results:

```
Choose an option:
'1' - for Trapezoidal Rule
'2' - for Simpson's Rule

What is your choice?: 3
Invalid Input!
Choose an option:
'1' - for Trapezoidal Rule
'2' - for Simpson's Rule

What is your choice?:
```

- **Trapezoidal Rule and Simpson's Rule :** Non-numeric Values
  Results:

```
Choose an option:
'1' - for Trapezoidal Rule
'2' - for Simpson's Rule

What is your choice?: 1
Trapezoidal Rule chosen
Enter value of c for y = e^(dt)*sin(ct) function: a
Enter value of d for y = e^(dt)*sin(ct) function: b
Invalid Input!
Enter value of c for y = e^(dt)*sin(ct) function:

Choose an option:
'1' - for Trapezoidal Rule
'2' - for Simpson's Rule

What is your choice?: 2
Simpson's Rule chosen
Enter value of c for y = e^(dt)*sin(ct) function: a
Enter value of d for y = e^(dt)*sin(ct) function: b
Invalid Input!
Enter value of c for y = e^(dt)*sin(ct) function:
```

- **Simpson's Rule:** Odd number of intervals

```
Choose an option:
'1' - for Trapezoidal Rule
'2' - for Simpson's Rule

What is your choice?: 2
Simpson's Rule chosen
Enter value of c for y = e^(dt)*sin(ct) function: 1
Enter value of d for y = e^(dt)*sin(ct) function: 1
Enter lower limit for the integration: 1
Enter upper limit for the integration: 2
Enter number of intervals: 3
Interval must be even.
Enter number of intervals:
```

- **Limits :** Non-numeric Values and lower limit greater than upper limit Results:

```
Choose an option:
'1' - for Trapezoidal Rule
'2' - for Simpson's Rule

What is your choice?: 1
Trapezoidal Rule chosen
Enter value of c for y = e^(dt)*sin(ct) function: 1
Enter value of d for y = e^(dt)*sin(ct) function: 1
Enter lower limit for the integration: a
Enter upper limit for the integration: b
Invalid Input!
Enter lower limit for the integration:

Choose an option:
'1' - for Trapezoidal Rule
'2' - for Simpson's Rule

What is your choice?: 1
Trapezoidal Rule chosen
Enter value of c for y = e^(dt)*sin(ct) function: 1
Enter value of d for y = e^(dt)*sin(ct) function: 2
Enter lower limit for the integration: 4
Enter upper limit for the integration: 3
Lower limit should be less than upper limit.
Enter lower limit for the integration:
```